

Reviewer Pack — Minimal Hodge-theoretic Norm and Resolution Proof Width Corr...

Entry 4f2cdbb607c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 13:51:49 UTC

Minimal Hodge-theoretic Norm and Resolution Proof Width Correlation

Entry ID: 4f2cdbb607c4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 13:51:49 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For every CNF φ with n variables, the minimal Hodge-theoretic norm of the associated vector space $V(\varphi)$ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\|V(\varphi)\|_H = \Theta(w(\varphi))$

Rationale (proposer's reasoning):

The Hodge-theoretic norm can capture geometric properties of complex algebraic varieties, which may reflect the complexity of the satisfiability problem in a way that is independent of standard algebraic operations.

Taxonomy category: Algebraic Geometry / Boolean Satisfiability (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7b264283491505d4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the Hodge-theoretic norm $\|V(\varphi)\|_H$ and resolution proof width $w(\varphi)$ of random CNFs φ with $n \leq 40$ variables exceeds 0.7, across at least 100 independent seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge theory" AND "resolution proof complexity" - "minimal Hodge-theoretic norm" AND CNF - "Boolean satisfiability" AND resolution proof width AND Hodge theory

Top relevant hits considered: - [http://arxiv.org/abs/1510.02682v8] Classical and Quantum Algorithms for the Boolean Satisfiability Problem - [http://arxiv.org/abs/1602.06867v4] Lower bound for the Complexity of the Boolean Satisfiability Problem - [http://arxiv.org/abs/1605.07503v7] A novel algorithm for solving the Decision Boolean Satisfiability Problem without algebra

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_cnf(n):
    cnf = []
    for _ in range(2 * n):
        clause = set()
        while len(clause) < 3:
            lit = random.choice([-i, i] for i in range(1, n + 1))
            if lit not in clause and -lit not in clause:
                clause.add(lit)
        cnf.append(tuple(sorted(clause)))
    return cnf

def hodge_norm(cnf):
    m = len(cnf)
    n = max(abs(lit) for clause in cnf for lit in clause)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for clause in cnf:
        for lit in clause:
            A[abs(lit)][abs(lit)] += 1
    det = determinant(A, n + 1)
    return abs(det)

def determinant(matrix, n):
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        det += ((-1) ** j) * matrix[0][j] * determinant(minor(matrix, 0, j), n - 1)
```

```

    return det

def minor(matrix, i, j):
    return [row[:j] + row[j+1:] for row in matrix[1:]]

def resolution_width(cnf):
    stack = []
    assignment = {}
    for clause in cnf:
        if all(lit not in assignment for lit in clause):
            stack.append(clause)
        else:
            resolved = False
            while stack and not resolved:
                top_clause = stack.pop()
                for lit in top_clause:
                    if -lit in assignment:
                        resolved = True
                        break
            if resolved:
                new_assignment = {**assignment, -lit: True}
                new_clause = [l for l in top_clause if l != -lit]
                if not all(lit in new_assignment for lit in new_clause):
                    stack.append(new_clause)

    return len(stack)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    metrics = []

    for n in n_values:
        cnf = generate_cnf(n)
        hodge_norm_value = hodge_norm(cnf)
        width_value = resolution_width(cnf)
        metrics.append((hodge_norm_value, width_value))

    if len(metrics) >= 30:
        break

    if len(metrics) < 10:
        return {
            "metric_name": "Hodge Norm vs Resolution Width",
            "metric_value": None,
            "instances_tested": len(metrics),
            "n_max": max(n_values[:len(metrics)]),
            "conjecture_holds": False,
            "counterexample": "Not enough instances tested"
        }

    hodge_norms = [m[0] for m in metrics]
    widths = [m[1] for m in metrics]
    mean_hodge_norm = sum(hodge_norms) / len(hodge_norms)
    mean_width = sum(widths) / len(widths)
    correlation = (sum((h - mean_hodge_norm) * (w - mean_width) for h, w in zip(hodge_norms, widths)) /
                   (len(metrics) * (sum((h - mean_hodge_norm) ** 2 for h in hodge_norms) / len(hodge_norms))))

```

```

        sum((w - mean_width) ** 2 for w in widths) / len(widths))) if len(metrics) > 0

    return {
        "metric_name": "Hodge Norm vs Resolution Width",
        "metric_value": correlation,
        "instances_tested": len(metrics),
        "n_max": max(n_values[:len(metrics)]),
        "conjecture_holds": correlation >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {'seed': {seed}, **result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"Not enough instances tested\" first_failed={r['seed']}")
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7726e9a.py", line 118, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7726e9a.py", line 77, in run_trial
    cnf = generate_cnf(n)
         ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7726e9a.py", line 23, in generate_cnf
    lit = random.choice([-i, i] for i in range(1, n + 1))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/usr/lib/python3.12/random.py", line 346, in choice
    if not len(seq):
           ^^^^^^^

```

TypeError: object of type 'generator' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Pearson correlation coefficient required to support the conjecture. | next: Investigate and fix the error in the test code to generate the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12629
2	propose	ollama_remote	glm4:latest	0	0	12291
3	propose	ollama_remote	glm4:latest	0	0	17589
4	preregistration	ollama_remote	glm4:latest	0	0	9781
5	novelty	ollama_remote	glm4:latest	0	0	8744
6	novelty	ollama_remote	glm4:latest	0	0	9678
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30564
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13773
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11473
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14365
11	judge	ollama_remote	glm4:latest	0	0	11586

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 152473 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/4f2cdbb607c4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4f2cdbb607c4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4f2cdbb607c4.tar.gz` (if generated)